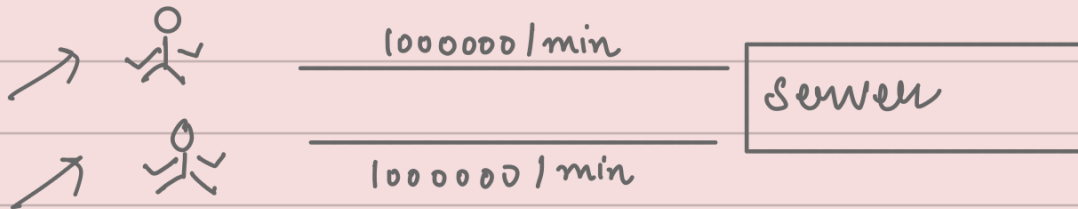
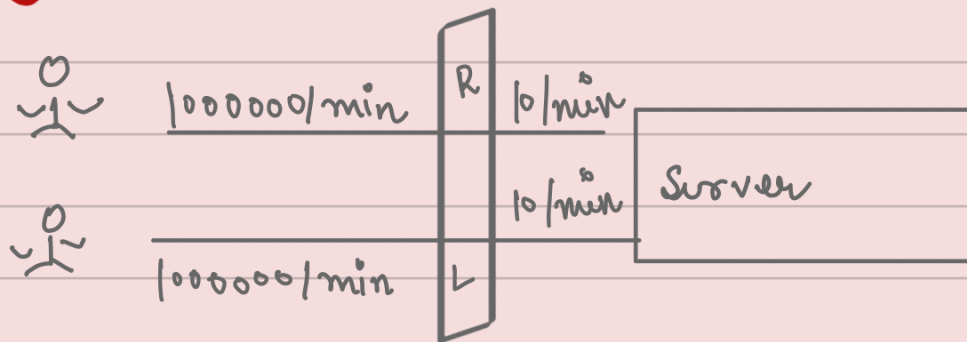


RATE LIMITER

A Rate limiter limits the numbers of client requests allowed to be sent over a specific period. If the API request count exceeds the threshold defined by the Rate limiter, all excess calls are blocked.



{ HAVE A BREAK,
HAVE A KITKAT }

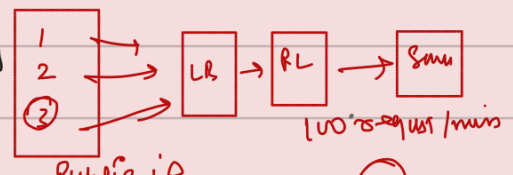


- 1) Prevent Overloading of server
- 2) Protect from Dos/DDOS attack
- 3) Fair Resource allocation

→ Advantages

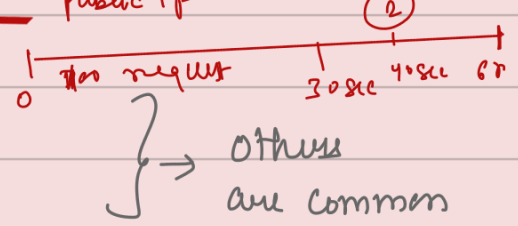
- 1) Increased latency
- 2) Potential for false positive
- 3) Excessive Blocking → (IP)

→ Disadvantages



PROBLEM REQUIREMENTS

- 1) Make Rate limiter
- 2) low latency



Capacity Estimates

- 1) 1 Billion Users
- 2) 20 Services to ratelimit
- 3) UserID = 8 bytes, Counter = 4 bytes
- 4) 1 billion $\times$ 20 $\times$ (8+4) = 240 Gb

Concepts

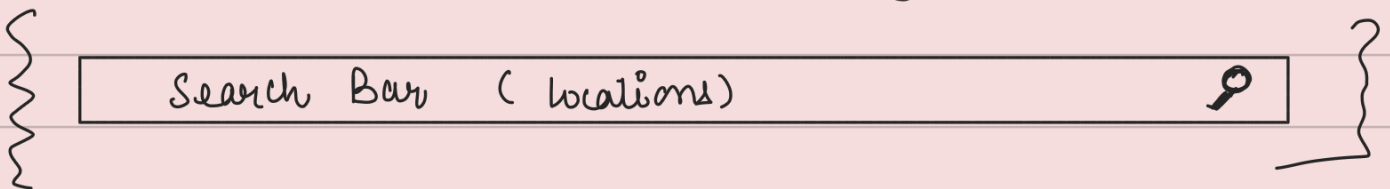
(★) Rate limit on UserID or IP Address?

→ User Id

- Easy to track, Bad users are blocked
- [works only when logged in.]
- what about search features on website?

→ IP Address

- No need to signed on.
- will it give false positive.



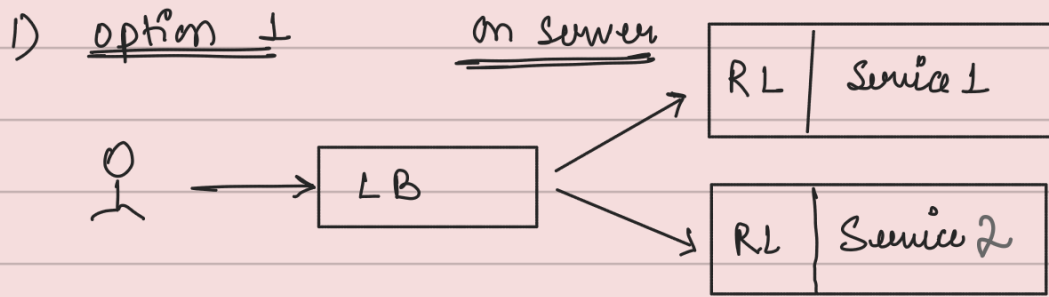
→ hybrid

- IP → not authenticated
- UserID → authenticated

(★) Interface

boolean ratelimiter (long uuid,
 string ipaddress,
 string service,
 Datetime Request)

(★) Where to put Rate limiter?



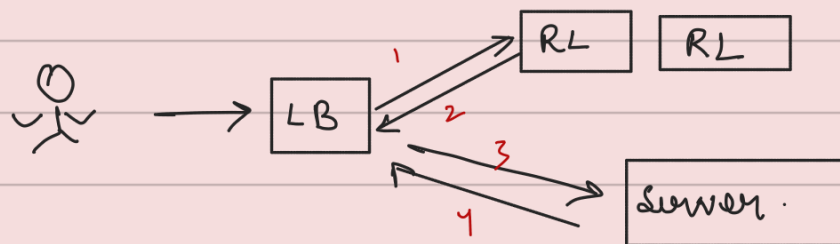
Pro :-

1) no api call (fast)

Cons :-

- 1) tightly coupled (they cannot be scaled separately)
- 2) server network Bandwidth

2) option 2 Dedicate Distributed Rate limiter



Pro

- 1) shield Servers
- 2) scale independently (up/down)

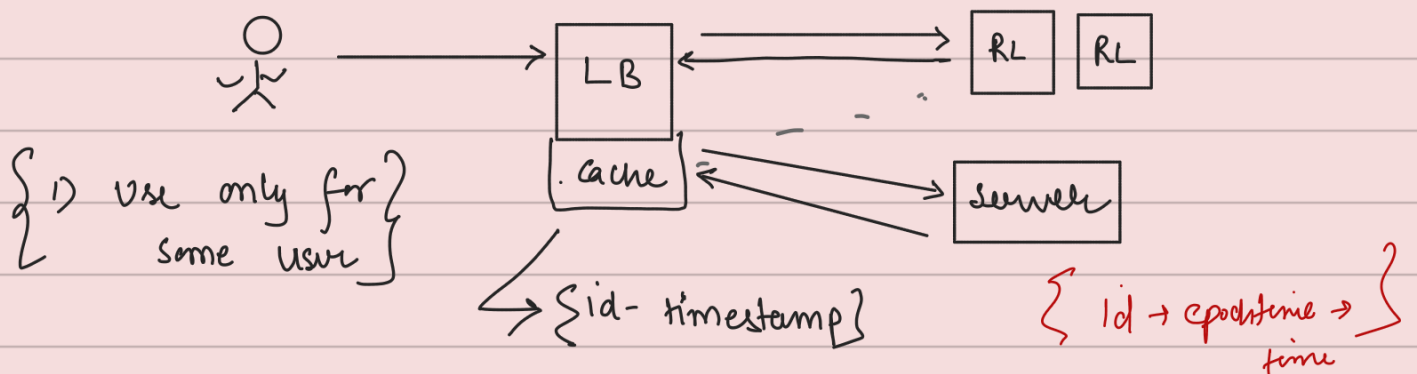
Cons

1) Latency (Extra Api call)

3) option 3

Rate limiting caching

Use write back cache on load balancer



DB Choice

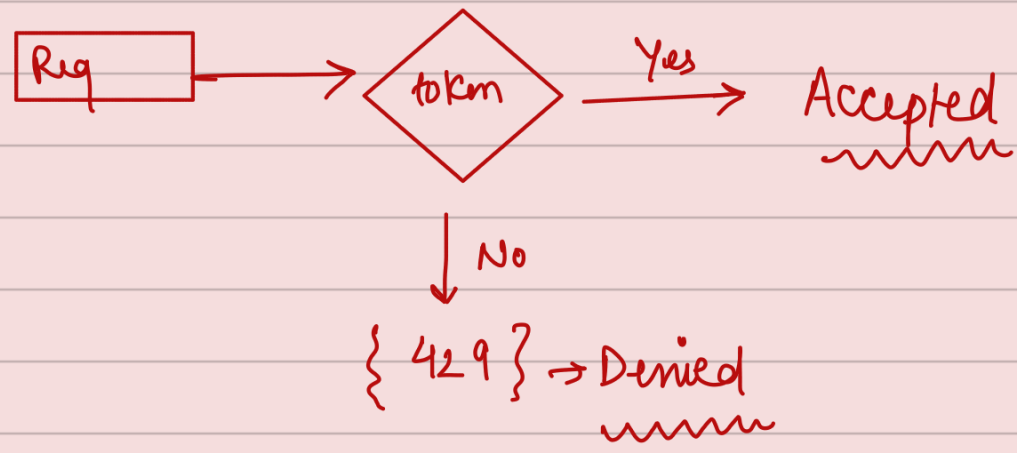
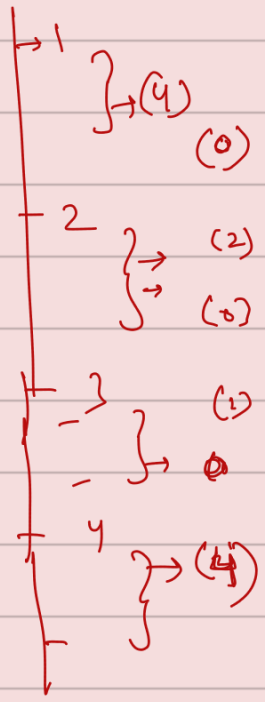
1) Redis

1) we need something very fast

Algorithms

1) Token Bucket :-

- 1) Capacity = 4 token
- 2) Refill rate
→ property to add 2 token in a min



Pros

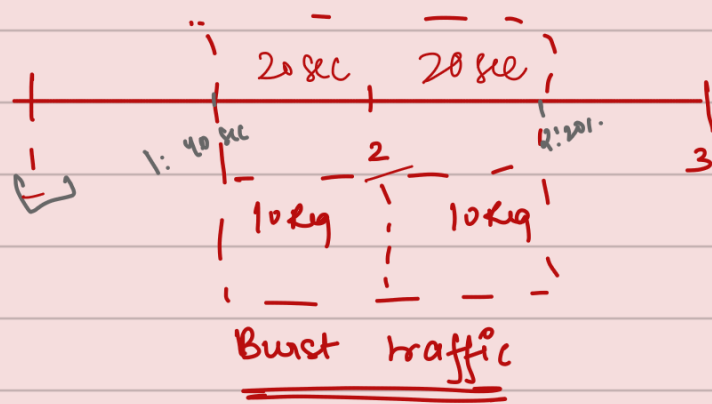
1) Allow occasional burst traffic

Cons

1) (same)

Capacity = 10

ex 10 token every min $\rightarrow$ Refill

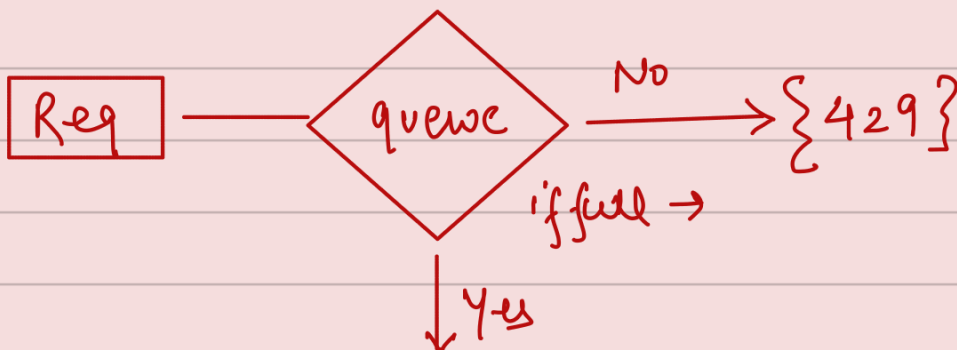


1) (Same)

2) token wastage

2) Leaking Bucket

- 1) implemented through queue
- 2) Request process on Constant Rate



$\rightarrow$ [uses FIFO]

Pros

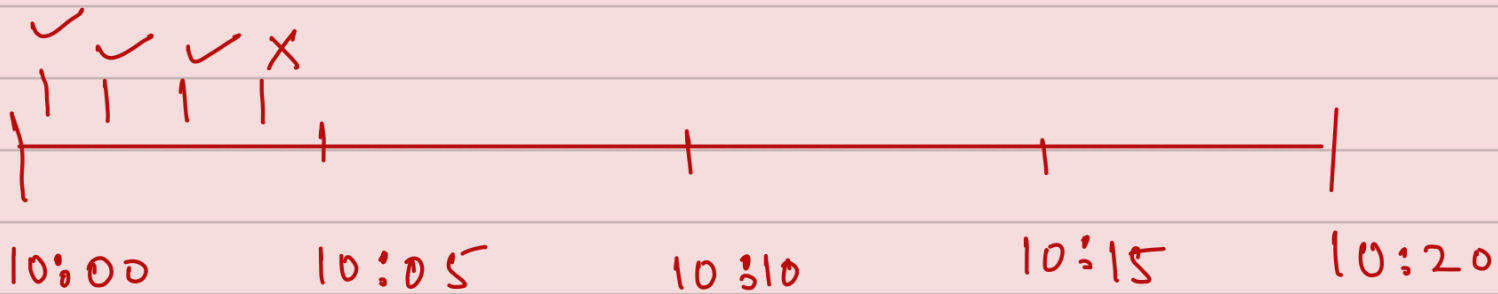
- 1) predictable
- 2) constant behaviour
- 3) Good for [1) youtube Streaming
2) Video Call]

Cons

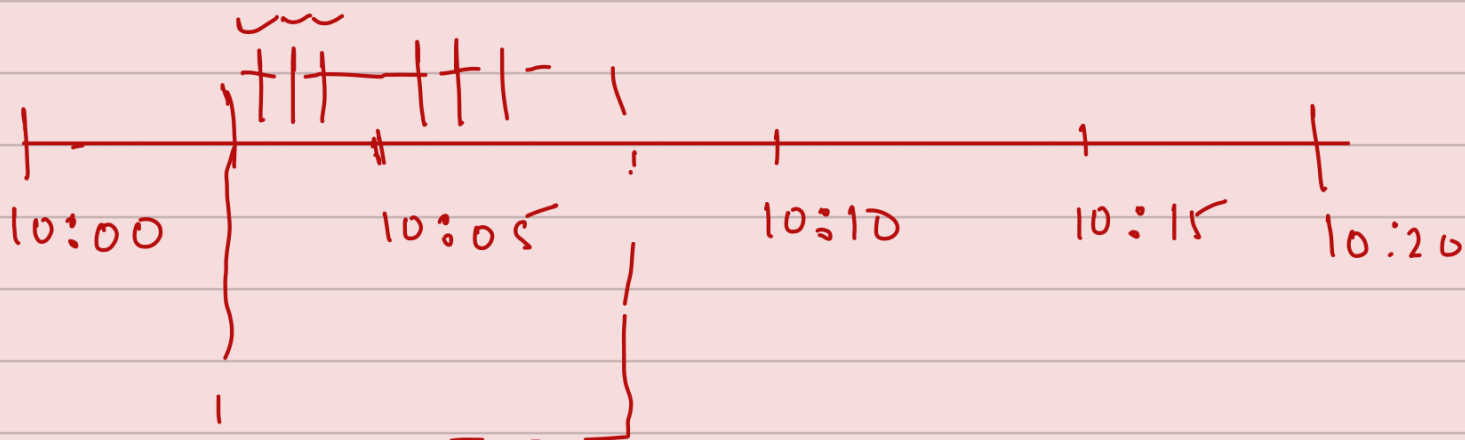
1) lets play? [CSK vs MI]

will not work → [RCB VS CSK] on Burst traffic

3) fixed Window Counter



assume counter = 3



{ <within 1 min>
many request } → Burst traffic

Dont × forget × Redis

Code

bool ratelimit(userid , service name, Requesttime)

{

current = redis.get(<key>)

[ex - servicename userid]

```

if (current["maxtime"] ≤ request time)
    → current["counter"] ++ ;

```

```

else current = (gettimeRange(request time)
    > 1)

```

```

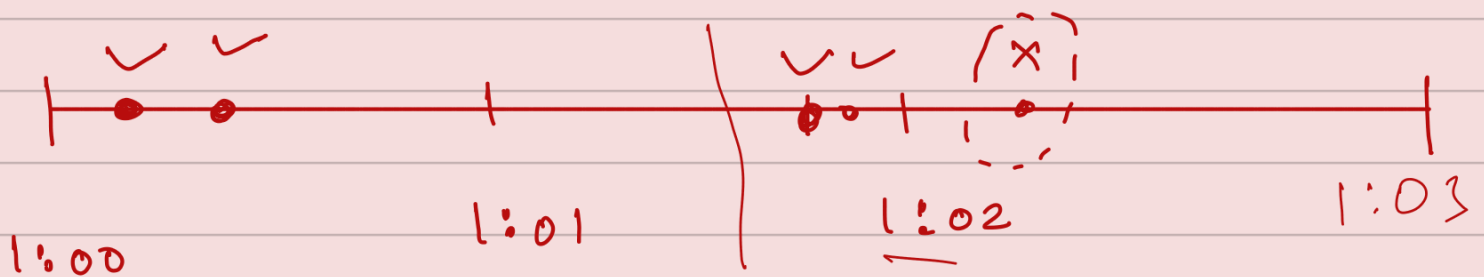
return current["count"] > limit ;

```

4) Sliding Window Counter

fixed window
counter

sliding
window log.



Do you know [linked list?]
[capacity = 2]

1) 1:00 } <head>

2) 1:00 — 1:00:20 } → increase linked list.

→ 1:01:40 - 1:00 →
(remove) ⇒ (100 sec)

1:01:40 - 1:00:10

4) 1:01:40 1:01:50

6) 1:01:40 1:01:50 $\leftarrow$ 1:02:10

No entry!

$$[\text{Capacity} = 2]$$

Code :-

```
bool RateLimit ( userID, serviceName, requestTime)
```

```
request window = redis.get(< Key > < array >);  
ex → service name_userid
```

$$\text{cutoff time} = \text{request time} - \text{window size} \cdot \text{load}()$$

```
while ( augustWindow.hasNext() & !corrected )
```



```

    requestwindow.pop();
}

```

result = requestwindow.size() > limit

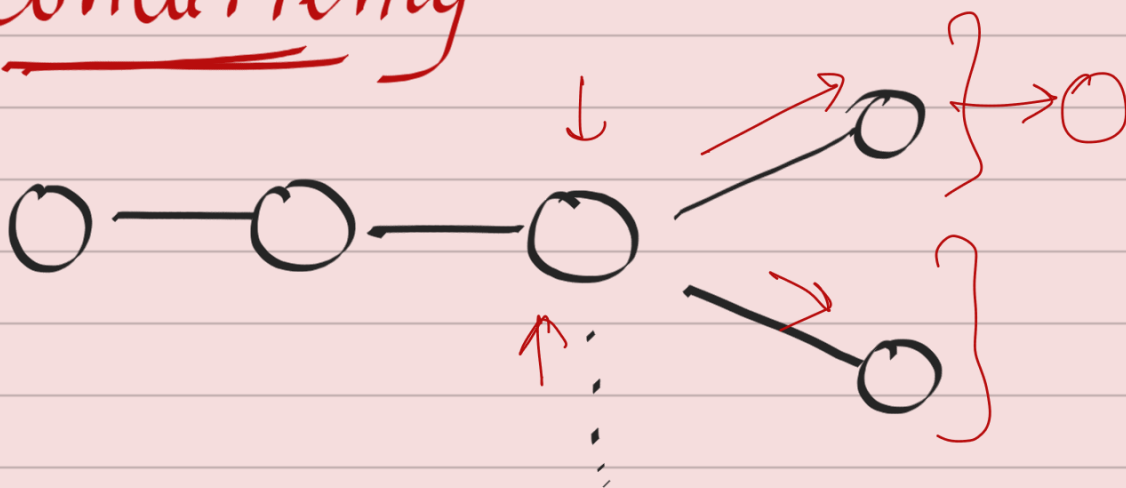
```

if result == false { requestwindow.push
                     (requesttime) }
return result;

```

}

Concurrency



{ same time }

- 1) Need locking
- 2) Concurrent linked list.....

Design

Redis



